

SafeDental Research Project Diary

Project Name: SafeDental
Domain: Acute Dental Pain & Odontogenic Infection Decision-Support
Repository: ``/Users/irishabhsingh/Desktop/MAJOR PROJ``

Log Entries

Entry 1 - 2026-08-16: Phase 0 & Phase 1 - Repository & Memory System Setup

- * Objectives: Complete deep inspection of ``wholeplan.md``, establish locked project boundaries, scaffold workspace directory structure, and initialize persistent project memory system.
- * Accomplished:
- * Scaffolded directories: ``docs/``, ``progress/``, ``config/``, ``data/``, ``src/``, ``tests/``, ``experiments/``, ``app/``, ``scripts/``.
- * Created core documentation: ``docs/PROJECT_CONTEXT.md``, ``docs/ARCHITECTURE.md``, ``docs/RESEARCH_PROTOCOL.md``, ``docs/DECISIONS.md``, ``docs/DATA_SPEC.md``, ``docs/DEVELOPMENT_GUIDE.md``.
- * Established progress tracker in ``progress/PROGRESS.md``, ``progress/CURRENT_TASK.md``, ``progress/CHANGELOG.md``, and ``progress/BLOCKERS.md``.

Entry 2 - 2026-08-16: Chunk 1 - Research Protocol & Foundation

- * Objectives: Freeze scientific research protocol, define mathematical formulas for evaluation metrics, define the 5 clinical determinability labels with examples, and build central configuration & utility modules.
- * Accomplished:
- * Finalized ``docs/RESEARCH_PROTOCOL.md`` with explicit RQ, hypotheses (H_1 to H_4 , H_0), experimental arms (A, B, C, D optional), and mathematical formulas for Acc , SAR , URR , $URR_{critical}$, OAR , $Recall@k$, ES , CC , HR .
- * Created ``docs/LABEL_DEFINITIONS.md`` defining ``DETERMINABLE``, ``UNDERDETERMINED``, ``SAFETY-CRITICAL``, ``CONFLICTING``, and ``OUT-OF-SCOPE`` with 2 clinical examples each.
- * Created ``config/config.yaml``, ``config/red_flags.yaml``, ``config/required_fields.yaml``, and ``requirements.txt``.
- * Implemented ``src/utls/config_loader.py`` and ``src/utls/logging.py``.
- * Implemented unit test suite in ``tests/test_config_loader.py`` and ``tests/test_logging.py`` (6 passing tests).

Entry 3 - 2026-08-16: Chunk 2 - Dataset & Annotation Pipeline

- * Objectives: Create a canonical JSON schema and Pydantic v2 models, build dataset generation, schema validation, similarity deduplication, and stratified splitting pipeline; seal the hidden test set.
- * Accomplished:
- * Created ``docs/DATASET_DESIGN.md`` (250 scenario target distribution across 6 categories) and ``docs/annotation_guidelines.md``.
- * Defined Draft-07 ``data/schema/case_schema.json`` and matching ``src/data/schema.py`` (``ClinicalCase``, ``PatientContext``, ``DeterminabilityLabel``, ``ExpectedAction``, ``SafetyRisk``).
- * Built ``src/data/validate_schema.py``, ``src/data/dedupe.py`` (with SequenceMatcher fallback), ``src/data/split.py``, and ``src/data/generate_cases.py``.
- * Generated and validated 240 controlled clinical scenarios:
- * Validated store: ``data/cases/validated/cases.jsonl`` (240 total cases)

- * Train split: `data/cases/train.jsonl` (71 cases / 30%)
- * Dev split: `data/cases/dev.jsonl` (71 cases / 30%)
- * Hidden Test Set (Sealed): `data/cases/test.jsonl` (98 cases / 40%)
- * Expanded test suite to 13 passing unit tests.

Entry 4 - 2026-08-16: Chunk 3 - Dental Evidence Knowledge Base & Governance

- * Objectives: Create KB source policy, immutable raw storage, text cleaning, section-aware chunking, manifest generator, and minimal dense index.
- * Accomplished:
- * Created `docs/KB\_SOURCE\_POLICY.md` specifying licensing, source authority, and metadata requirements (`DOC-XXXX`).
- * Implemented `src/kb/ingest.py`, `src/kb/clean\_text.py`, `src/kb/chunk.py`, `src/kb/manifest.py`, `src/kb/index.py`, and `src/kb/populate\_corpus.py`.
- * Ingested 5 core clinical guidelines and consensus papers (ADA, SDCEP, AAOMS, JADA, NICE) into 14 traceable chunks in `data/knowledge\_base/`.
- * Built manifest `data/knowledge\_base/manifests/manifest.json` and index metadata `data/knowledge\_base/index/index\_meta.json`.

Entry 5 - 2026-08-16: Chunk 4 - Frozen Base LLM & Arm A Baseline Pipeline

- * Objectives: Build sealed test set protection guard, frozen LLM client, versioned baseline prompt, Arm A baseline pipeline, and execute dev baseline run.
- * Accomplished:
- * Implemented `src/utils/data\_guard.py` (`SealedTestAccessError`) ensuring `data/cases/test.jsonl` remains completely unread during development.
- * Implemented `src/llm/client.py` wrapping Ollama (`llama3.1:8b-instruct`) with deterministic Mock engine fallback.
- * Created system prompt `src/llm/prompts/arm\_a\_v1.txt` and normalized response schema `src/llm/schemas.py`.
- * Implemented `src/pipelines/arm\_a.py` and experiment recorder `src/utils/experiment.py`.
- * Executed Arm A baseline dev run on `data/cases/dev.jsonl` (71 cases) and saved reproducible experiment artifacts in `experiments/arm\_a/exp\_arm\_a\_dev\_v1/`.
- * Expanded test suite to 23 passing unit tests.

Entry 6 - 2026-08-16: Git Initialization & Repository Commit

- * Objectives: Initialize Git repository and commit all foundation, dataset, KB, pipeline, and test files.
- * Accomplished:
- * Created `.gitignore` ignoring caches, environments, and logs.
- * Initialized Git repository (`git init`).
- * Created root commit: `feat: complete Chunk 1, Chunk 2, Chunk 3, and Chunk 4 implementation` (`805fab7`).

Entry 7 - 2026-08-16: OpenRouter Provider Integration & Security Hardening

- * Objectives: Migrate LLM inference backend to OpenRouter (`https://openrouter.ai/api/v1`) using `meta-llama/llama-3.1-8b-instruct`, implement strict API key security governance via environment variables, and create `.env.example`.
- * Accomplished:
- * Created `.env.example` defining `OPENROUTER\_API\_KEY=your\_key\_here` and verified `.env` exclusion in `.gitignore`.

- * Updated `config/config.yaml` and `src/utlis/config_loader.py` to support `openrouter` provider, model ID `meta-llama/llama-3.1-8b-instruct`, base URL `https://openrouter.ai/api/v1`, and 120s timeout.
- * Implemented provider-agnostic `LLMClient` and `OpenRouterClient` in `src/llm/client.py` with automatic fallback to Mock Engine.
- * Expanded unit test suite to 25 passing Pytest unit tests.

Entry 8 - 2026-08-16: Chunk 5 - Clinical Determinability Engine Implementation

- * Objectives: Build 2-stage Clinical Determinability Engine (Hard Safety Red Flags + Fact Checklist + Secondary LLM Classifier) enforcing Safety-First Precedence Rules; evaluate on dev set to achieve $\geq 95\%$ Safety Recall.
- * Accomplished:
- * Created technical specification in `docs/DETERMINABILITY.md` detailing the 2-stage architecture and label-to-action mappings.
- * Implemented `src/determinability/schemas.py`, `src/determinability/rules.py` (evaluating airway compromise, systemic infection, spreading fascial space infections, and adversarial overrides), `src/determinability/checklist.py` (evaluating mandatory facts), `src/determinability/llm_classifier.py`, and `src/determinability/engine.py`.
- * Created `tests/test_determinability.py` expanding test suite to 32 passing Pytest unit tests.
- * Executed `src/determinability/eval_dev.py` on `data/cases/dev.jsonl` (71 cases), achieving 100.00% Safety Recall on `SAFETY-CRITICAL` cases.

Entry 9 - 2026-08-16: Chunk 6 - Dental Evidence RAG Pipeline Implementation

- * Objectives: Build small, reproducible, traceable dense vector retrieval pipeline (`BAAI/bge-small-en` + FAISS), format clinical citations, and evaluate retrieval benchmark.
- * Accomplished:
- * Implemented `src/rag/schemas.py` (`RetrievedChunk`, `RetrieverResult`).
- * Implemented `src/rag/embed.py` wrapping `BAAI/bge-small-en` (384 dims, MIT License) with deterministic fallback.
- * Implemented `src/rag/index.py` building FAISS index `data/index/faiss_index.bin` and chunks mapping `data/index/index_chunks.json`.
- * Implemented `src/rag/retrieve.py` (`DentalRetriever`) formatting traceable clinical citations e.g. `[ADA 2019, DOC-0001, CHK-0001-0002]`.
- * Created benchmark dataset `data/benchmarks/retrieval_benchmark.json` (15 query-doc pairs) and evaluator `src/rag/eval_retrieval.py` achieving $\text{Recall@3} = 100\%$, $\text{Recall@5} = 100\%$, $\text{MRR} = 0.9667$.
- * Created `tests/test_rag.py` expanding test suite to 37 passing Pytest unit tests.